
PHASEGATE: Phase-Aware CPU Retrieval Scheduling for On-Device LLMs on Unified Memory

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 On-device assistants increasingly run GPU-based LLM inference alongside CPU
2 tasks such as retrieval, indexing, and tool execution on the same unified-memory
3 system. Under a saturated local-retrieval workload, four concurrent searches raise
4 95th-percentile (p95) decode latency by 60–61% on two M4 systems, whereas pre-
5 fill latency rises by only 5.7–6.9%. A conventional scheduler applies one retrieval-
6 concurrency limit throughout an LLM request; that limit must remain safe during
7 sensitive decode and therefore leaves CPU capacity unused during safer prefill.
8 On our base-M4 configuration, one concurrent search satisfies the latency limits,
9 but two do not. PHASEGATE instead calibrates separate limits for prefill and de-
10 code, selecting four and one, respectively, on this configuration. It achieves $2.0\times$
11 the latency-constrained retrieval throughput of the best tested feasible fixed policy
12 while meeting both latency limits in every evaluation run. A phase-blind control
13 replays a calibration-derived wall-clock schedule with the same two limits but
14 never observes LLM phase; it achieves similar raw retrieval throughput yet vio-
15 lates the output-token limit in every run. M2 and M2 Pro Mac minis reproduce
16 the policy ordering, while output-length sweeps show that the advantage narrows
17 as decode occupies more of each request.

18 1 Introduction

19 Compact on-device models include Apple’s foundation model for Apple Intelligence and Google’s
20 Gemini Nano [1, 2]. Yet an assistant also performs retrieval, contextual-memory lookup, indexing,
21 parsing, and tool execution on the CPU. Representative agentic applications spend substantial time
22 in CPU-side tools [3], and on-device vector-search systems must coordinate interactive queries with
23 background ingestion and index maintenance [4].

24 Server accelerators stream model state from GPU-local high-bandwidth memory (HBM), separate
25 from host DRAM used by CPU tools [5, 6]. Apple Silicon instead exposes CPU and GPU to unified
26 memory [7], making CPU pointer chasing and GPU weight streaming compete for memory service.
27 Prior integrated-SoC work throttles CPU traffic [8], but does not ask *when* overlap is safer.

28 LLM phase supplies that signal: prefill processes prompt tokens in parallel and reuses weight loads,
29 whereas decode emits one token at a time and repeatedly streams model state.

30 To instantiate CPU-side retrieval, we use FAISS with a Hierarchical Navigable Small World
31 (HNSW) index for approximate nearest-neighbor search [9, 10]. Vector retrieval supports retrieval-
32 augmented generation by finding stored items similar to a query and adding them to the LLM con-
33 text [11]. HNSW performs irregular graph traversal, but our saturated queue is a capacity test, not a
34 complete agent replay; we separately test 5%, 25%, and 100% retrieval-active time. On a fanless M4
35 MacBook Air, four HNSW searches increase p95 time-to-first-token (TTFT), our prefill-sensitive

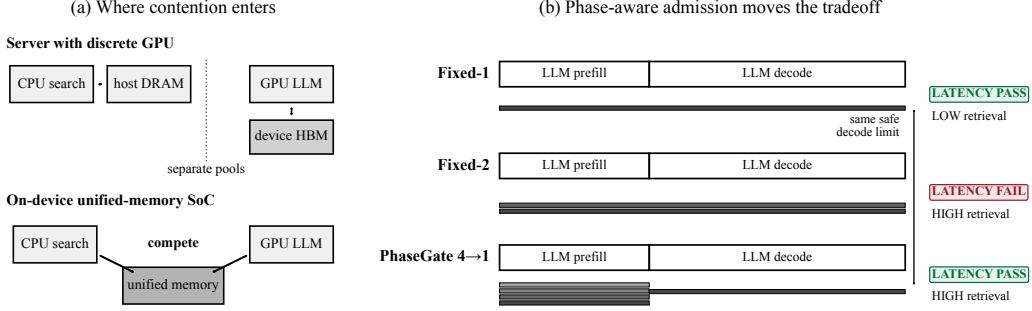


Figure 1: (a) A server’s CPU search and GPU LLM use separate memory pools, whereas their on-device counterparts share unified memory. (b) On the measured base-M4 configuration, explicit PASS/FAIL labels show the tradeoff: Fixed-1 meets both latency limits but restricts retrieval, while Fixed-2 retrieves more but violates the output-token limit. Calibrated PHASEGATE 4→1 combines high retrieval with the safe decode limit by using four-way concurrency only during prefill. Schematic.

metric, by 5.7%, but time-per-output-token (TPOT), our decode metric, by 61%; a fan-cooled base-M4 Mini preserves the direction. Section 2 defines this saturated workload and shows why CPU byte rate alone misses the risk.

A conventional retrieval pool uses one maximum concurrency throughout an LLM request. We write Fixed- n for this phase-oblivious policy with at most n simultaneous searches in both phases, and $p \rightarrow d$ for a PHASEGATE policy allowing up to p searches during prefill and d during decode. On the primary M4, Fixed-1 meets both latency limits, while the faster Fixed-2 violates output-token latency. PHASEGATE instead calibrates separate prefill and decode limits (Figure 1) and selects 4→1: four searches during prefill, but one during decode. This pair is deployment-specific, and we compare it with a phase-blind timer that alternates the same limits without observing LLM phase.

Server LLM systems separate prefill and decode across GPU pools, while shared-memory controls throttle co-running CPU traffic or yield after detecting contention [12, 6, 8, 13]. PHASEGATE instead leaves LLM placement unchanged and uses the known phase boundary as an anticipatory admission signal for independent CPU retrieval; the full comparison appears in Appendix B.

Contributions. (1) We establish prefill–decode asymmetry using HNSW on two M4 systems and add a decode-only access-pattern control on the M4 Air. (2) We design PHASEGATE to preserve a safe decode limit while using higher prefill concurrency; a time-matched phase-blind control isolates phase alignment. (3) We evaluate the scheduler on M4, M2, and M2 Pro Minis, vary output length, and synthetically sweep the fraction of time with retrieval demand.

2 Observation: Phase-Asymmetric Interference

Experimental Setup. We co-run Qwen2.5-1.5B at 4-bit using MLX-LM v0.31.3 [14, 15] with a read-only FAISS HNSW index (100,000 384-dimensional vectors; graph connectivity $M = 32$); LLM requests use 2,048 input and 128 output tokens. For the comparisons reported below, we run two or four concurrent single-threaded FAISS HNSW queries, corresponding to FIXED-2 and FIXED-4. We keep the query queue nonempty during active periods to measure the maximum sustainable retrieval throughput under an LLM-latency constraint. This is a capacity experiment for waiting batched or cross-session searches, not a typical single-user trace or prerequisite retrieval that finishes before its own request. We separately insert synthetic idle periods while retaining a backlog whenever retrieval demand is active.

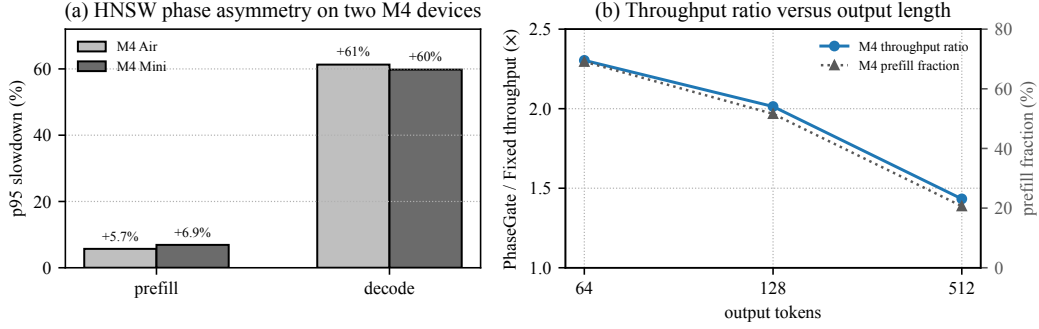


Figure 2: (a) Four concurrent HNSW searches show the same p95 asymmetry on a fanless M4 Air and fan-cooled base-M4 Mini; each Mini bar is the median of three runs with 300 LLM requests per run. (b) On the base-M4 Mini, PHASEGATE/Fixed throughput falls with output length (five pairs per length); the right axis shows the M4 prefill fraction.

Decode Is More Sensitive to Concurrent Retrieval. On the M4 Mini, we first measure LLM latency without retrieval, then co-run either two or four concurrent HNSW searches. Each concurrency setting uses three runs of 300 LLM requests, with the two settings evaluated in randomized order. TTFT measures prompt processing; TPOT measures output-token generation. With two concurrent searches on the M4 Mini, p95 TTFT increases by only 1.3%, whereas p95 TPOT increases by 31%. With four, Figure 2a shows 5.7% versus 61% on the Air and 6.9% versus 60% on the Mini; the Mini shows greater decode sensitivity in every repetition. This pattern is consistent with prefill amortizing weight reads while decode has little reuse.

Access Pattern Matters Too. After establishing the phase asymmetry, we isolate access pattern by fixing the hardware to the M4 Air and examining decode only. An address-jumping random gather reports just 2.4 GB/s of logical bandwidth but increases p95 decode latency by 29%, whereas a contiguous 51 GB/s scan increases it by 27%. Thus byte rate alone is a poor interference proxy: irregular pointer chasing can be more disruptive than a much higher-rate sequential scan, which is relevant to graph-based retrieval such as HNSW [10].

3 PHASEGATE

The goal is not merely to admit more work, but to admit it during safer prefill and restore a calibrated latency-safe limit for decode. PHASEGATE uses LLM phase, rather than CPU utilization or byte rate, to decide when a queued CPU workload such as retrieval may start.

Policy and Calibration. Let p and d be the prefill and decode concurrency limits. A phase-oblivious scheduler uses one limit throughout (e.g., $p = d = 1$); PHASEGATE permits $p > d$. The deployment supplies a baseline-relative budget B ; $B = 1.25$, for example, allows p95 TPOT and TTFT to rise by at most 25% over their no-retrieval baselines. Calibration tests candidate (p, d) pairs and selects the highest retrieval queries per second (QPS) among those that consistently meet both limits. Material changes to hardware, model, sequence lengths, retrieval, or load require recalibration.

Runtime. MLX-LM emits prefill and decode events; PHASEGATE admits a queued CPU workload (e.g., a search) only while the active count is below that phase’s limit. Already-running CPU workloads are not preempted: when the phase limit decreases, they finish while new work waits until the active count falls below the new limit. We request no core affinity or thread-priority (QoS) override.

4 Experiments

Experimental Setup. The scheduler experiments reuse Section 2’s base-M4 Mini, model, HNSW index, and 2,048-input/128-output-token requests while adding Fixed caps, separate PHASEGATE phase caps, and the phase-blind TimeGate control. We measure CPU throughput and p95 TTFT/TPOT under a backlogged queue; Appendix A lists the full hardware and software setup.

4.1 Main Experiments

We calibrate FIXED-0, FIXED-1, FIXED-2, FIXED-4, and PHASEGATE 2→0, PHASEGATE 2→1, PHASEGATE 4→0, PHASEGATE 4→1, and PHASEGATE 4→2 in three randomized 100-request runs per candidate. A candidate is eligible only if both p95 latency metrics remain at or below the baseline-relative budget B in every run. Across budgets 10–45% above baseline, $B = 1.25$ is the tightest point at which a CPU-performing Fixed policy and a PHASEGATE policy are both eligible; it is a controlled comparison point, not a universal user threshold.

Table 1: Base-M4 calibration sweep at $B = 1.25$. Arrows show the preferred direction; values are the worst of three calibration runs, and unrounded values determine eligibility. All worst TTFT ratios are < 1.1 ; bold policies are selected for held-out evaluation.

Fixed cap	CPU throughput $\uparrow$ (kQPS)	Worst GPU TPOT $\downarrow$ ($\times$ baseline)	PHASEGATE caps	CPU throughput $\uparrow$ (kQPS)	Worst GPU TPOT $\downarrow$ ($\times$ baseline)
0	0	1.0	2→0	0.79	1.3
1	0.78	1.2	2→1	1.1	1.2
2	1.4	1.4	4→0	1.2	1.3
4	2.1	1.7	4→1	1.6	1.2
			4→2	1.8	1.4

Table 1 exposes the central tradeoff. Raising a constant cap from one to two or four increases CPU throughput from 0.78 to 1.4 or 2.1 kQPS, but worst TPOT rises to $1.4\times$ or $1.7\times$ baseline and exceeds the $1.25\times$ budget. Separating the phases admits PHASEGATE 4→1: it reaches 1.6 kQPS while holding worst TPOT to $1.2\times$; raising its decode cap to two reaches 1.8 kQPS but again violates the budget at $1.4\times$. Calibration therefore selects FIXED-1 and PHASEGATE 4→1, the highest-throughput eligible policy in each family.

We hold those choices fixed and evaluate them in seven paired randomized comparisons with 250 LLM requests per policy. FIXED-1 and PHASEGATE 4→1 meet both latency limits in all seven runs, but PHASEGATE 4→1 reaches 1.6 kQPS versus 0.78 kQPS by increasing concurrency only during prefill (Table 2).

4.2 Additional Analysis

The main comparison shows that phase-specific caps unlock CPU throughput, but it does not by itself establish why the gain occurs or how broadly it persists. We therefore add targeted analyses for phase alignment, intermittent demand, device and output-length sensitivity, and boundary-transition overhead while keeping the selected policies unchanged.

Phase-Blind Control. Higher average concurrency alone could explain the throughput increase, so we designed a phase-blind control called TimeGate. It replays the calibration-derived wall-clock schedule with the same high and low limits, high-limit time fraction, and switching rate as PHASEGATE, but never observes LLM phase. TimeGate reaches a similar 1.5 kQPS, yet raises p95 TPOT by 60% and meets both latency limits in 0/7 runs: its high cap overlaps 23% of decode, compared with 0.0030% for PHASEGATE. Thus the additional CPU work must be aligned with the actual LLM phase rather than merely time-matched.

Table 2: Base-M4 evaluation at $B = 1.25$. Latency, throughput, and transition rows report medians across seven paired randomized repetitions with 250 requests per policy; “7/7” means both latency limits pass in all seven.

Metric	FIXED-1	PHASEGATE 4→1	TimeGate 4/1
Runs meeting both limits	7/7	7/7	0/7
p95 TPOT increase (%)	17	18	60
p95 TTFT increase (%)	0.61	4.6	4.3
CPU throughput (kQPS)	0.78	1.6	1.5
p95 transition gap (ms)	18	23	24

Intermittent Retrieval Demand. A continuously backlogged queue is a capacity test, whereas practical retrieval demand may be intermittent. We therefore make retrieval active for 5%, 25%, or 100% of wall-clock time; the 5% and 25% cases use 2.0-s backlogged bursts separated by mean idle gaps of 38 s and 6.0 s, and each duty level uses five matched three-policy comparisons. The median full-wall-clock PHASEGATE/Fixed throughput ratio remains $2.0\times$ at all three duties, with rounded 95% paired-bootstrap intervals [1.8, 2.0], [2.0, 2.1], and [2.0, 2.0]. FIXED-1 and PHASEGATE each meet both limits in all 15 runs, while TimeGate passes none; at 5% duty, idle time lowers the absolute throughput to 79 versus 40 QPS without erasing the ratio.

Output Length and Cross-Device Replication. To test whether the result is tied to one chip or one prefill/decode balance, we retain the selected caps while varying output length and repeating the policy comparison on other Mac minis. On M4, PHASEGATE/Fixed throughput falls from $2.3\times$ at 64 tokens to $2.0\times$ at 128 and $1.4\times$ at 512 as prefill’s share of LLM processing time falls from 69% to 52% and 21% (Figure 2b). The M2 Mini shows the same decreasing direction, from $1.8\times$ at 128 tokens to $1.4\times$ at 512 (see Appendix A.1); devices are analyzed separately and the association is descriptive. The M2 Pro comparison yields $2.5\times$, but lacks TimeGate and pageout-free pairs, so it supports only the policy ordering.

Transition and Memory Costs. Because CPU work is non-preemptive, work admitted near the phase boundary may continue briefly into decode. An in-memory monotonic clock records token and phase timestamps without a monitoring process, allowing us to measure the gap from prefill completion to the first decoded token and two tail token-gap metrics. The 5.2-ms p95 transition penalty occurs once per request and is 0.29% of baseline TTFT; both tail token-gap metrics rise by less than 1%. No swap occurs and memory pressure remains normal; pageouts remain a caveat, but removing any matched comparison still yields $2.0\times$.

Validity Checks. Fresh no-retrieval baselines vary by about 1.0% within the continuous and duty-sweep M4 studies. Swap growth, abnormal pressure, thermal warnings, corrupt traces, incorrect policy behavior, or severe drift invalidate a run, with one retry; small system-wide pageouts remain warnings. All planned policy runs yield valid measurements, and invalid attempts remain in the artifact.

5 Limitations and Conclusion

Limitations. The full protocol uses one base-M4 Mini; narrower tests use one each of M2 Mini, M2 Pro Mini, M4 Air, and M1 Air. All use one 1.5B 4-bit LLM and 2,048-token prompts; scheduler and HNSW tests use read-only HNSW and device-specific caps. The duty sweep uses saturated bursts, not random arrivals or full agent workflows; protocol/pageout differences preclude cooling or paging-neutrality claims. A future latency-aware controller could extend the phase signal by reducing CPU limits after unexpected token slowdown, but robust trigger and recovery rules remain open. **Conclusion.** Across the three synthetic duties, PHASEGATE delivers $2.0\times$ the median latency-safe throughput of Fixed-1; TimeGate fails, confirming that phase alignment matters here. Code, traces, and scripts will be released.

References

- [1] Tom Gunter et al. Apple Intelligence foundation language models. *arXiv preprint arXiv:2407.21075*, 2024.
- [2] Gemini Team et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [3] Ritik Raj, Souvik Kundu, Ishita Vohra, Hong Wang, and Tushar Krishna. Towards understanding, analyzing, and optimizing agentic AI execution: A CPU-centric perspective. *arXiv preprint arXiv:2511.00739*, 2025.
- [4] Xinkui Zhao, Qingyu Ma, Yifan Zhang, Hengxuan Lou, Sai Liu, Chang Liu, Guanjie Cheng, Naibo Wang, and Yueshen Xu. MUSE: A heterogeneity-aware multimedia search engine for mobile SoCs. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26)*, 2026. doi: 10.1145/3767308.3835897. arXiv:2511.19192.
- [5] NVIDIA. NVIDIA H100 Tensor Core GPU architecture. White paper, NVIDIA, 2022.
- [6] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210, 2024.
- [7] Apple. Apple unleashes M1. Apple Newsroom, 2020. URL <https://www.apple.com/newsroom/2020/11/apple-unleashes-m1/>.
- [8] Waqar Ali and Heechul Yun. Protecting real-time GPU kernels on integrated CPU–GPU SoC platforms. In *30th Euromicro Conference on Real-Time Systems (ECRTS)*, volume 106 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 19:1–19:22, 2018. doi: 10.4230/LIPIcs.ECRTS.2018.19.
- [9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.
- [10] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- [11] Apple. Managing the on-device foundation model’s context window. Apple Developer Documentation, Technical Note TN3193, 2026. URL <https://developer.apple.com/documentation/Technotes/tn3193-managing-the-on-device-foundation-model-s-context-window>.
- [12] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024. doi: 10.1109/ISCA59077.2024.00019.
- [13] Tong Xin, Xinrui Shi, Mingkai Dong, and Zeyu Mi. Inference in the shadows: Taming memory bandwidth contention in mobile LLM inference with Sereno. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 2299–2318, 2026.
- [14] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [15] MLX Community. MLX-LM: Run LLMs with MLX. Software, version 0.31.3, 2026. URL <https://github.com/ml-explore/mlx-lm/tree/v0.31.3>.
- [16] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput–latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134, 2024.
- [17] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19274–19286, 2023.
- [18] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 539–558, 2022.
- [19] Om Mohite. FusionML: Prefill, not decode—mechanism and boundaries of CPU+GPU co-execution on unified-memory Apple Silicon. *arXiv preprint arXiv:2607.22785*, 2026.
- [20] Taehwan Park, Geonho Lee, and Min-Soo Kim. MobileRAG: A fast, memory-efficient, and energy-efficient method for on-device RAG. *arXiv preprint arXiv:2507.01079*, 2025.
- [21] Tianxi Lu and Sherief Reda. Agentic CPU–GPU scheduling for heterogeneous AI workloads. *arXiv preprint arXiv:2607.22242*, 2026.
- [22] Yifei Wang, Hancheng Ye, Yechen Xu, Cong Guo, Chiyue Wei, Qinsi Wang, Dongting Li, Tingjun Chen, Hai Li, Danyang Zhuo, and Yiran Chen. MARS: Efficient, adaptive co-scheduling for heterogeneous agentic systems. *arXiv preprint arXiv:2604.26963*, 2026.

218 A Experimental Details (Appendix)

Table A1: Fan-cooled scheduler devices and shared setup (P/E CPU cores; UMA unified memory).

M4 primary	Mac mini (Mac16,10); 4P+6E CPU, 10-core GPU, 16 GB UMA
M2 replication	Mac mini (Mac14,3); 4P+4E CPU, 10-core GPU, 16 GB UMA
M2 Pro earlier	Mac mini (Mac14,12); 6P+4E CPU, 16-core GPU, 16 GB UMA
Runtime	macOS 26.5/26.6.1 (continuous study/duty sweep); MLX 0.31.2; MLX-LM 0.31.3
Workload	2,048 input/128 output tokens; 100k 384-D vectors; queue kept nonempty

Table A2: Mac mini evaluations (two significant digits); safe runs meet both limits, and “—” denotes an unevaluated control.

Device	B	PHASEGATE/Fixed throughput ($\times$)	PHASEGATE/TimeGate throughput ($\times$)	Fixed safe runs	PHASEGATE safe runs	TimeGate safe runs
Base-M4 Mini	1.25	2.0	1.0	7/7	7/7	0/7
Base-M2 Mini	1.25	1.8	1.0	5/5	5/5	0/5
M2 Pro Mini	1.10	2.5	—	7/7	7/7	—

219 A.1 M2 Output-Length Replication

220 The M2 Mini uses its separately calibrated baseline and the same fixed policies at both output lengths.
 221 The median PHASEGATE/Fixed throughput ratio decreases from $1.8\times$ across five pairs at 128 output
 222 tokens to $1.4\times$ across three pairs at 512 tokens.

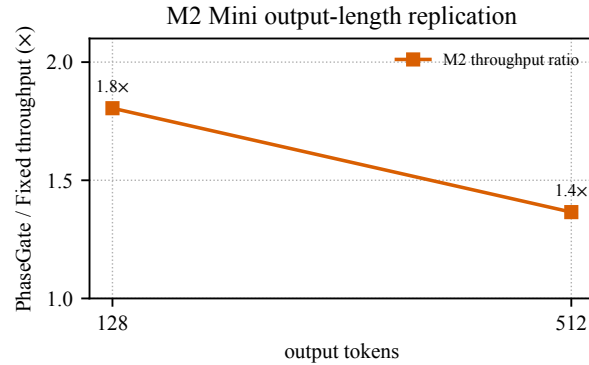


Figure A1: M2 Mini output-length replication with unchanged PHASEGATE 4 $\rightarrow$ 1 and FIXED-1 caps. The 128-token point summarizes five pairs and the 512-token point summarizes three; M2 is analyzed separately from M4.

B Related Work (Appendix)

Phase-Structured LLM Serving. Server systems exploit the different resource profiles of pre-fill and decode inside the inference stack. Splitwise and DistServe assign the phases to separately provisioned GPU pools, Sarathi-Serve chunks prefills to avoid stalling ongoing decodes, and speculative decoding reduces sequential decode work [12, 6, 16, 17]. PHASEGATE is complementary: it leaves LLM execution unchanged on one on-device GPU and uses the phase boundary to regulate an independent CPU retrieval workload.

Shared-Memory Interference. BWLOCK++ protects integrated-GPU kernels by throttling co-running CPU memory traffic, while Sereno detects mobile memory contention and yields background LLM inference to foreground applications [8, 13]. Adjacent GPU-sharing work such as REEF instead provides kernel preemption and controlled co-execution [18]. FusionML divides pre-fill operators across the Apple-Silicon CPU and GPU, whereas PHASEGATE schedules separate retrieval work and treats LLM phase as an anticipatory victim-side signal [19]. A feedback guard could complement that signal when interference falls outside offline calibration.

On-Device Retrieval. Apple’s Foundation Models guidance presents retrieval-augmented generation as a way to fetch relevant context for an on-device model [11]. MobileRAG reduces vector-search and context-processing costs on mobile devices, and MUSE coordinates interactive vector queries with background index maintenance on mobile SoCs [20, 4]. FAISS and HNSW provide established approximate-nearest-neighbor mechanisms for this retrieval kernel [9, 10]. These systems motivate our workload, but do not make a continuously backlogged queue representative of a single-user trace.

Agentic CPU–GPU Scheduling. Recent studies characterize CPU-heavy tool execution, map heterogeneous tools across CPUs and GPUs, or co-schedule multi-turn LLM–tool workflows [3, 21, 22]. They optimize end-to-end heterogeneous workflows, primarily on server platforms. To our knowledge, prior work does not compare fixed, phase-aligned, and time-matched phase-blind retrieval limits on an on-device unified-memory system. PHASEGATE asks the narrower question of how pending CPU retrieval should follow prefill and decode while preserving TTFT and TPOT; evaluation with end-to-end agent traces remains future work.